DEPARTMENT OF ECE, SVIT

DIGITAL SYSTEM DESIGN USING VERILOG

BEC302

MODULE 1

PRINCIPLES OF COMBINATIONAL LOGIC

Syllabus: Definition of combinational logic, Canonical forms, Generation of switching equations from truth tables, Karnaugh maps (2, 3, 4 variables), Quine-McCluskey Minimization Technique, Quine-McCluskey using Don't Care Terms.

Prepared by: Pavithra G S, Asst. Professor
pavithra.gs@saividya.ac.in

TABLE OF CONTENTS

1. Combinational Logic – Introduction & Boolean Algebra

1.1 What is Combinational Logic?

A logic circuit can be classified into two fundamental categories:

- Combinational Logic Circuits
- Sequential Logic Circuits

A combinational logic circuit contains only logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) and does NOT contain any storage or memory elements such as flip-flops or latches. The output of a combinational circuit depends solely and entirely on the present combination of input values — there is no dependency on past inputs or history.

A sequential logic circuit, in contrast, contains storage elements (flip-flops, registers, latches) in addition to logic gates. Memory elements are connected back to the combinational logic to form a feedback path, which means the output depends on both the current inputs and the present state stored in memory.

Combinational Circuit	Sequential Circuit
`Inputs → [Logic Gates Only] → Outputs` No memory. Output = f(current inputs only).	`Inputs → [Logic Gates + Memory] → Outputs` Has memory. Output = f(current inputs + present state).

Key properties of Combinational Logic Circuits:

- Output at any instant depends only on the combination of inputs at that instant.
- No feedback path — information does not loop back from output to input.
- No clock signal is required for operation.
- Examples: Adders, Subtractors, Multiplexers, Decoders, Encoders, Comparators.
- Applications: Calculators, computers, digital measuring instruments, digital communication systems, musical instruments, automatic control of industrial machines.

1.2 Laws of Boolean Algebra

Boolean algebra was developed by George Boole and is the foundation of digital logic design. The basic laws of Boolean algebra are similar to ordinary algebra but adapted for binary variables (0 and 1).

Law	Addition Form	Multiplication Form
Commutative Law	A + B = B + A	A · B = B · A
Associative Law	A + (B + C) = (A + B) + C	A(BC) = (AB)C
Distributive Law	A(B + C) = AB + AC	(A + B)(A + C) = A + BC

These laws allow us to rearrange and simplify Boolean expressions without changing their logical meaning. The commutative law means the ORDER of variables does not matter. The associative law means GROUPING does not matter. The distributive law allows FACTORING of expressions — vital for minimization.

1.3 Rules of Boolean Algebra (Rules 1–12)

In addition to the three basic laws, twelve fundamental rules govern Boolean algebra. These rules form the toolkit for all Boolean simplification:

Rule	Expression	Verbal Description
1	A + 0 = A	OR with 0 gives the variable unchanged (Identity for OR)
2	A + 1 = 1	OR with 1 always gives 1 (Annihilator for OR)
3	A · 0 = 0	AND with 0 always gives 0 (Annihilator for AND)
4	A · 1 = A	AND with 1 gives the variable unchanged (Identity for AND)
5	A + A = A	OR of a variable with itself equals itself (Idempotent)
6	A + A' = 1	A variable OR its complement always equals 1 (Complement)
7	A · A = A	AND of a variable with itself equals itself (Idempotent)
8	A · A' = 0	A variable AND its complement always equals 0 (Complement)
9	(A')' = A	Double complement returns the original variable (Involution)
10	A + AB = A	Absorption: adding a product that already contains A is redundant
11	A + A'B = A + B	Simplification: A OR (A'B) simplifies to A OR B
12	(A+B)(A+C) = A+BC	Factoring: distribution works in both directions

Proof of Rule 10: A + AB = A

This rule states that if A appears in a sum, any product term that ALSO contains A can be dropped.

```
A + AB = A(1 + B)      [Distributive Law]
       = A · 1         [Rule 2: 1 + B = 1]
       = A             [Rule 4: A · 1 = A]   ✓
```

Proof of Rule 11: A + A'B = A + B

This rule is extremely useful in K-map and algebraic simplification.

```
A + A'B = (A + A'B) + A'B        [Rule 10: A = A + AB]
        = (AA + AB) + A'B          [Rule 7: A = AA]
        = AA + AB + AA' + A'B      [Rule 8: adding AA'=0]
        = (A + A')(A + B)              [Factoring]
        = 1·(A + B)                [Rule 6: A + A' = 1]
```

```
= A + B                                    [Rule 4]   ✓
```

Proof of Rule 12: (A + B)(A + C) = A + BC

```
(A+B)(A+C) = AA + AC + AB + BC     [Distributive Law]
           = A + AC + AB + BC       [Rule 7: AA = A]
           = A(1 + C) + AB + BC     [Factoring]
       = A + AB + BC          [Rule 2: 1+C=1, then A·1=A]
           = A(1 + B) + BC          [Factoring]
       = A + BC                 [Rule 2 and Rule 4]   ✓
```

1.4 DeMorgan's Theorems

DeMorgan's Theorems are among the most important tools in digital logic. They allow conversion between AND and OR operations using complementation — essential for NAND/NOR logic implementation.

> Theorem 1: The complement of a PRODUCT equals the SUM of individual complements.(XY)' = X' + Y'Theorem 2: The complement of a SUM equals the PRODUCT of individual complements.(X + Y)' = X' · Y'

Verification of Theorem 1 by Truth Table:

X	Y	XY	(XY)'	X'+Y'	Match?
0	0	0	1	1+1=1	✓
0	1	0	1	1+0=1	✓
1	0	0	1	0+1=1	✓
1	1	1	0	0+0=0	✓

Generalised DeMorgan's Theorem for n variables:

```
(A · B · C · ... · N)' = A' + B' + C' + ... + N'
(A + B + C + ... + N)' = A' · B' · C' · ... · N'
```

Worked Example using DeMorgan's:

Simplify: F = ((AB)' + C)'

```
F = ((AB)' + C)'
= (AB)'' · C'          [DeMorgan Theorem 2]
= AB · C'              [Double complement, Rule 9]
= ABC'                 [Final simplified form]
```

> Tip for VTU: DeMorgan's Theorems are used to implement functions using only NAND gates or only NOR gates — a very common exam topic!

2. Canonical & Standard Forms

Every Boolean function can be expressed in two canonical (standard) forms: Sum of Products (SoP) and Product of Sums (PoS). Understanding these forms is fundamental to logic minimization.

2.1 Min Terms and Max Terms

For n Boolean variables, we get 2^n combinations. Each combination corresponds to exactly one Minterm and one Maxterm.

Minterm (m): A product (AND) term in which every variable appears exactly once, either in true or complemented form. A minterm evaluates to 1 for exactly ONE combination of inputs.

Maxterm (M): A sum (OR) term in which every variable appears exactly once, either in true or complemented form. A maxterm evaluates to 0 for exactly ONE combination of inputs.

Rule for writing minterms and maxterms:

- In a MINTERM: if the variable value is 0, write the variable COMPLEMENTED; if 1, write it TRUE.
- In a MAXTERM: if the variable value is 0, write the variable TRUE; if 1, write it COMPLEMENTED.
- Minterm m_i and Maxterm M_i are complements of each other: m_i = (M_i)'

2-Variable Minterm / Maxterm Table:

Index	x	y	Minterm (m)	Maxterm (M)
0	0	0	m0 = x'y'	M0 = x + y
1	0	1	m1 = x'y	M1 = x + y'
2	1	0	m2 = xy'	M2 = x' + y
3	1	1	m3 = xy	M3 = x' + y'

3-Variable Minterm / Maxterm Table:

Idx	A	B	C	Minterm	Maxterm
0	0	0	0	m0 = A'B'C'	M0 = A+B+C
1	0	0	1	m1 = A'B'C	M1 = A+B+C'
2	0	1	0	m2 = A'BC'	M2 = A+B'+C
3	0	1	1	m3 = A'BC	M3 = A+B'+C'
4	1	0	0	m4 = AB'C'	M4 = A'+B+C
5	1	0	1	m5 = AB'C	M5 = A'+B+C'

Idx	A	B	C	Minterm	Maxterm
6	1	1	0	m6 = ABC'	M6 = A'+B'+C
7	1	1	1	m7 = ABC	M7 = A'+B'+C'

Key fact: If there are n Boolean variables, there are exactly 2^n minterms and 2^n maxterms. Minterm m_i = (Maxterm M_i)' — they are complements of each other.

2.2 Canonical SoP Form (Sum of Min Terms)

The Canonical SoP (Sum of Products) form is obtained by ORing together all the minterms for which the output function is 1. This is also called the MINTERM CANONICAL FORM.

Every product term in canonical SoP contains ALL n literals (either true or complemented). This makes it unique for a given function.

Steps to write Canonical SoP:

1. Construct the truth table for the function.
2. Identify all rows where the output F = 1.
3. Write the minterm for each such row (use complemented variable for 0-input, true variable for 1-input).
4. OR all identified minterms together.

Notation: F = Σm(i, j, k, ...) where i,j,k are the minterm indices where F=1.

2.3 Canonical PoS Form (Product of Max Terms)

The Canonical PoS (Product of Sums) form is obtained by ANDing together all the maxterms for which the output function is 0. This is also called the MAXTERM CANONICAL FORM.

Every sum term in canonical PoS contains ALL n literals. Steps to write Canonical PoS:

5. Identify all rows in the truth table where output F = 0.
6. Write the maxterm for each such row (use true variable for 0-input, complemented variable for 1-input).
7. AND all identified maxterms together.

Notation: F = ΠM(i, j, k, ...) where i,j,k are the maxterm indices where F=0.

Important: SoP and PoS forms represent the same function. F = Σm(a,b,...) = ΠM(all other indices). They are duals of each other.

2.4 Standard SoP Form

The Standard SoP form is the SIMPLIFIED version of the Canonical SoP form. Product terms need NOT contain all n literals — they represent groups of minterms. The Standard SoP form is what you get after K-map or algebraic simplification.

Steps to obtain Standard SoP from Canonical SoP:

8. Start with the Canonical SoP form.
9. Apply Boolean postulate x + x = x to duplicate required terms.

10. Use the Distributive Law to group and factor terms.
11. Apply x + x' = 1 to cancel complementary literals.
12. Simplify using x·1 = x.

Worked Example: Convert f = p'qr + pq'r + pqr' + pqr to Standard SoP form.

Step 1: Duplicate pqr twice (since x + x = x): f = p'qr + pq'r + pqr' + pqr + pqr + pqr

Step 2: Group: f = qr(p' + p) + pr(q' + q) + pq(r' + r)

Step 3: Apply x + x' = 1: f = qr(1) + pr(1) + pq(1)

Step 4: Apply x·1 = x: f = qr + pr + pq

Final Standard SoP: f = pq + qr + pr

2.5 Standard PoS Form

The Standard PoS form is the simplified version of the Canonical PoS form. Sum terms need NOT contain all n literals.

Worked Example: Convert f = (p+q+r)(p+q+r')(p+q'+r)(p'+q+r) to Standard PoS form.

Step 1: Duplicate (p+q+r) twice: f = (p+q+r)(p+q+r)(p+q+r)(p+q+r')(p+q'+r)(p'+q+r)

Step 2: Use Distributive Law x+(yz) = (x+y)(x+z): f = (p+q+rr')(p+r+qq')(q+r+pp')

Step 3: Apply x·x' = 0: f = (p+q+0)(p+r+0)(q+r+0)

Step 4: Apply x+0 = x: f = (p+q)(p+r)(q+r)

Final Standard PoS: f = (p+q)(q+r)(p+r)

The Standard SoP and Standard PoS of the same function are DUALS of each other. f=pq+qr+pr (SoP) is dual to f=(p+q)(q+r)(p+r) (PoS).

3. Generation of Switching Equations from Truth Tables

A truth table completely specifies the behavior of a Boolean function. Given any truth table, we can systematically extract the Boolean expression using the canonical forms. This process is called 'generation of switching equations'.

3.1 Step-by-Step SoP Derivation

The procedure to derive the Sum of Products expression from a truth table:

13. Write out the complete truth table with all 2^n input combinations.
14. Identify every row where the output F = 1.
15. For each such row, write a minterm: for each input bit that is 0, write the complemented variable; for each input bit that is 1, write the true variable.
16. OR (sum) all identified minterms to get the Canonical SoP.
17. Optionally simplify using Boolean algebra or K-maps to get Standard SoP.

Memory Aid: For SoP → look for 1s in the output column. Write products (AND) of variables. For PoS → look for 0s in the output column. Write sums (OR) of variables.

3.2 Step-by-Step PoS Derivation

The procedure to derive the Product of Sums expression from a truth table:

18. Identify every row where the output F = 0.
19. For each such row, write a maxterm: for each input bit that is 0, write the TRUE variable; for each input bit that is 1, write the COMPLEMENTED variable.
20. AND (product) all identified maxterms to get the Canonical PoS.
21. Optionally simplify to get Standard PoS.

3.3 Worked Example – 3-Variable Function

Consider the following truth table for a 3-input, 1-output combinational function F(p,q,r):

Row	p	q	r	F	Minterm / Maxterm
0	0	0	0	0	M0 = p+q+r (maxterm)
1	0	0	1	0	M1 = p+q+r' (maxterm)
2	0	1	0	0	M2 = p+q'+r (maxterm)
3	0	1	1	1	m3 = p'qr (minterm)
4	1	0	0	0	M4 = p'+q+r (maxterm)
5	1	0	1	1	m5 = pq'r (minterm)
6	1	1	0	1	m6 = pqr' (minterm)
7	1	1	1	1	m7 = pqr (minterm)

From the truth table above:

- F = 1 at rows 3, 5, 6, 7 → Canonical SoP: F = Σm(3,5,6,7)
- F = 0 at rows 0, 1, 2, 4 → Canonical PoS: F = ΠM(0,1,2,4)

Writing out fully:

```
Canonical SoP: F = p'qr + pq'r + pqr' + pqr
Canonical PoS: F = (p+q+r)(p+q+r')(p+q'+r)(p'+q+r)
```

Verification: Both expressions represent the SAME function. One can verify by substituting any input combination and checking that both give the same output.

Both canonical forms describe the exact same Boolean function. The choice of which form to use depends on which leads to simpler minimization. Usually the form with fewer terms is preferred as the starting point.

4. Karnaugh Maps (2 to 4 Variables)

4.1 Introduction to K-Maps

Karnaugh (pronounced 'car-no') introduced the Karnaugh Map (K-Map) method in 1953 as a graphical tool for simplifying Boolean functions. It eliminates the tedious algebraic manipulation required in Boolean algebra simplification.

A K-Map is essentially a 2-dimensional grid version of the truth table, arranged so that adjacent cells differ in exactly ONE variable (Gray code ordering). This allows us to visually identify groups of 1s that can be combined to eliminate variables.

Properties of K-Maps:

- A K-Map for n variables has 2^n cells, one for each minterm.
- Adjacent cells differ in exactly ONE bit (Gray code adjacency).
- The K-Map wraps around — the leftmost and rightmost columns are adjacent, and the top and bottom rows are adjacent (toroidal topology).
- A group of 2^k cells (k=0,1,2,...) that are all 1s reduces k variables from the product term.
- Larger groups are always preferred — they give more simplification.

4.2 Two-Variable K-Map

For 2 variables (Y, Z), the K-Map has 2^2 = 4 cells:

Y\Z	0	1
0	m0	m1
1	m2	m3

Possible groupings in a 2-Variable K-Map:

- Group of 4 (all cells): eliminates both variables → F = 1
- Group of 2 (adjacent pairs): (m0,m1), (m2,m3), (m0,m2), (m1,m3) — eliminates 1 variable
- Group of 1 (single cell): no elimination, 2-literal product term

4.3 Three-Variable K-Map

For 3 variables (X, YZ), the K-Map has 2^3 = 8 cells. CRITICAL: The columns are ordered in Gray code: 00, 01, 11, 10 (NOT binary order 00,01,10,11). This ensures adjacency between columns 3 and 4.

X\YZ	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6

Adjacencies in 3-variable K-Map (wrap-around):

- m0 is adjacent to m1, m2, m4 (including wrap-around with m2)
- m2 and m0 are adjacent (columns 00 and 10 are adjacent due to Gray code)
- Groups of 4 possible: {m0,m1,m3,m2}, {m4,m5,m7,m6}, {m0,m1,m4,m5}, {m1,m3,m5,m7}, {m3,m2,m7,m6}, {m2,m0,m6,m4}
- A group of 4 reduces to a 1-literal term; a group of 8 (all) reduces to F=1

4.4 Four-Variable K-Map

For 4 variables (WX, YZ), the K-Map has 2^4 = 16 cells. Both rows and columns follow Gray code ordering:

WX\YZ	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

Wrap-around adjacencies in 4-variable K-Map:

- Row 00 and Row 10 are adjacent (top and bottom rows wrap).
- Column 00 and Column 10 are adjacent (leftmost and rightmost columns wrap).
- Corner cells m0, m2, m8, m10 form an adjacent group of 4 (all four corners).
- A group of 16 (entire map) gives F = 1; group of 8 gives 1-literal; group of 4 gives 2-literal; group of 2 gives 3-literal; group of 1 gives 4-literal.

4.5 Rules for K-Map Simplification

Follow these rules strictly to obtain the minimal sum of products expression:

22. Draw the appropriate K-Map and fill in 1s at positions corresponding to the minterms of the function. Fill don't-cares (d) where specified.
23. GROUPS MUST BE POWERS OF 2: 1, 2, 4, 8, 16. No other group sizes are valid.
24. ALL cells in a group must contain 1 (or don't-care d). You cannot include a 0 cell.
25. Groups must be RECTANGULAR in shape on the K-Map (including wrap-around rectangles).
26. LARGEST GROUPS FIRST: Always form the largest possible group for each 1. Larger groups eliminate more variables.
27. EVERY 1 must be covered by at least one group. No 1 may be left ungrouped.
28. Prefer groups that cover 1s not covered by any other group (Essential Prime Implicants).
29. Minimize the NUMBER of groups — use the fewest groups that cover all 1s.
30. Don't-care cells (d) may be included in a group if they help make it larger, but they MUST NOT be left as the only 1s in a group.
31. Each group produces one product term: the variable is retained if it is the SAME in all cells of the group, eliminated if it changes.

Essential Prime Implicant (EPI): A prime implicant that covers at least one minterm not covered by any other prime implicant. EPIs must ALWAYS be included in the final expression. Start by identifying EPIs, then add the minimum necessary additional PIs.

4.6 Don't Care Conditions

Don't care conditions (represented by 'd' or 'X' in K-Maps) arise in two situations:

- Input combinations that NEVER occur in practice (invalid states). For example, in a BCD counter only 0-9 are valid; states 10-15 never occur.
- Output values for which we genuinely DON'T CARE whether the circuit produces 0 or 1.

How to handle don't cares in K-Maps:

- Place 'X' in the K-Map cells corresponding to don't care minterms.
- You MAY include 'X' cells in a group if it makes the group LARGER (and thus gives more simplification).
- You MUST NOT include 'X' cells unless they help enlarge an existing group covering at least one 1.
- Don't care minterms are NEVER included in the PI chart (Phase 2 of QM) because they are not required outputs.

4.7 Worked Examples

Example 1 — 3-Variable K-Map: F(A,B,C) = Σm(0,2,4,5,6)

Step 1: Place 1s at positions 0,2,4,5,6 in the 3-variable K-Map:

A\BC	00	01	11	10
0	1(m0)	0	0	1(m2)
1	1(m4)	1(m5)	0	1(m6)

Step 2: Form groups:

- Group 1: {m0, m2, m4, m6} — 4 cells (A=0→1, B=0 stays, C=0 stays) → B'C' ... wait: m0=00,00; m2=01,10; m4=10,00; m6=11,10. Checking: B changes, C=0 fixed, A changes → C' (only C=0 is common) → Group gives: C'
- Group 2: {m4, m5} — 2 cells → AB' (A=1, B=0 fixed, C changes)

Simplified: F = C' + AB'

Example 2 — 4-Variable K-Map: F(A,B,C,D) = Σm(0,1,2,4,5,8,9,10,11)

Step 1: Place 1s at all listed minterm positions in the 4-variable K-Map.

AB\CD	00	01	11	10
00	1(m0)	1(m1)	0(m3)	1(m2)
01	1(m4)	1(m5)	0(m7)	0(m6)
11	0(m12)	0(m13)	0(m15)	0(m14)
10	1(m8)	1(m9)	1(m11)	1(m10)

Step 2: Form groups:

- Group 1: {m0,m1,m4,m5} — 4 cells in top-left → A'C' (A=0, B changes, C=0, D changes) → A'C'
- Group 2: {m0,m1,m8,m9} — 4 cells (top & bottom rows wrap in col 00,01) → B'C' (B=0, C=0)
- Group 3: {m8,m9,m10,m11} — 4 cells bottom row → AB' (A=1,B=0)
- Group 4: {m0,m2,m8,m10} — 4 corner-like cells → B'D' (B=0,D=0)

After checking coverage of all 1s: F = A'C' + AB' + B'D'

Always double-check: substitute a few minterms to verify your simplified expression gives F=1 at those points.

5. Quine-McCluskey (Tabulation) Method

5.1 Introduction & Need

The K-Map method, while intuitive and fast for small functions, becomes impractical for 5 or more variables because:

- It is difficult to visualize adjacencies in a large K-Map.
- There is no systematic way to guarantee the optimal grouping has been found.
- K-Maps cannot be easily automated by a computer.

The Quine-McCluskey (QM) method, also called the Tabulation Method, solves these problems. It was originally developed by W.V. Quine in 1952 and improved by E.J. McCluskey in 1956.

Advantages of the QM Method:

- Fully systematic and algorithmic — can be implemented by a computer.
- Guaranteed to find the minimal expression.
- Works for any number of variables.
- Handles don't care terms cleanly.
- Provides a verifiable, step-by-step procedure with clear decision criteria.

The QM method involves two main phases:

- Phase 1: Finding all Prime Implicants using the Implicant Table.
- Phase 2: Selecting the minimum cover using the Prime Implicant Chart.

5.2 Phase 1 – Step-by-Step Algorithm (Finding Prime Implicants)

PHASE 1: BUILDING THE IMPLICANT TABLE

Step 1: List all minterms where F=1 (also include don't care minterms) in binary representation.

Step 2: Count the number of 1-bits in each minterm's binary representation (called the 'index' or 'count of ones').

Step 3: Group minterms by their count of ones into groups G0, G1, G2, ... (ascending order).

Step 4: Compare each minterm in group G_k with every minterm in group G_{k+1}. Two minterms can be combined if they differ in EXACTLY ONE BIT POSITION. When combined, write the result with a dash '-' in the differing position and check/tick both minterms as 'used'.

Step 5: Repeat Step 4 for the newly generated implicants (size-2, size-4, etc.), continuing until no further combinations are possible. When comparing size-4 implicants, dashes must appear in the same positions.

Step 6: All implicants that were NOT checked/ticked (could not be combined further) are the PRIME IMPLICANTS.

IMPORTANT: When combining size-2 or larger implicants, the dash '-' positions must MATCH exactly. For example, -110 and -100 can combine (differ only in bit 2), giving -1-0. But -110 and 011- CANNOT combine because their dash positions differ.

5.3 Phase 2 – Prime Implicant Chart & Essential Prime Implicants

PHASE 2: SELECTING THE MINIMUM COVER

Step 1: Draw the Prime Implicant Chart. Rows = Prime Implicants (PIs). Columns = Original minterms (F=1 only, NOT don't cares).

Step 2: Mark an 'X' in the chart wherever a PI covers a minterm.

Step 3: Identify ESSENTIAL PRIME IMPLICANTS (EPIs): A column that has only ONE 'X' means that minterm can only be covered by that one PI. That PI is therefore ESSENTIAL — it must be included in the final expression.

Step 4: Select all EPIs. Mark all minterms covered by EPIs.

Step 5: If all minterms are covered by EPIs, you are done. If some minterms are still uncovered, select the minimum number of additional PIs needed to cover all remaining minterms (Petrick's method for complex cases).

Step 6: Write the final expression as the OR of all selected PIs.

Petrick's Method: For Step 5, if multiple options exist, form a boolean covering expression: each uncovered minterm m_i must be covered by at least one PI, i.e., (PI_a + PI_b + ...)(PI_c + PI_d + ...)... Expand and select the product with the fewest literals.

5.4 Fully Worked Example

Minimize: F(A,B,C,D) = Σm(4,8,10,11,12,15) + d(9,14)

Step 1: List minterms and don't cares in binary:

Minterm	Binary (ABCD)	# of 1s
m4 (F=1)	0100	1
m8 (F=1)	1000	1
m9 (d)	1001	2
m10 (F=1)	1010	2
m11 (F=1)	1011	3
m12 (F=1)	1100	2
m14 (d)	1110	3
m15 (F=1)	1111	4

Step 2: Group by number of 1s:

Group	Minterm	Binary	# 1s
G1	m4	0100	1
G1	m8	1000	1
G2	m9	1001	2

Group	Minterm	Binary	# 1s
G2	m10	1010	2
G2	m12	1100	2
G3	m11	1011	3
G3	m14	1110	3
G4	m15	1111	4

Step 3: Compare adjacent groups (Size-2 Implicants):

Pair	Binary Result	Notes
m(4,12)	0100 vs 1100 → -100	Differ in bit A
m(8,9)	1000 vs 1001 → 100-	Differ in bit D
m(8,10)	1000 vs 1010 → 10-0	Differ in bit C
m(8,12)	1000 vs 1100 → 1-00	Differ in bit B
m(9,11)	1001 vs 1011 → 10-1	Differ in bit C
m(10,11)	1010 vs 1011 → 101-	Differ in bit D
m(10,14)	1010 vs 1110 → 1-10	Differ in bit B
m(12,14)	1100 vs 1110 → 11-0	Differ in bit C
m(11,15)	1011 vs 1111 → 1-11	Differ in bit B
m(14,15)	1110 vs 1111 → 111-	Differ in bit D

Step 4: Compare size-2 implicants to get size-4 implicants (dashes must align):

Pair of Size-2	Binary Result	Notes
m(8,9)+m(10,11): 100- & 101-	10--	Differ in bit C → covers m8,9,10,11
m(8,10)+m(9,11): 10-0 & 10-1	10--	Same as above (duplicate)
m(8,10)+m(12,14): 10-0 & 11-0	1--0	Differ in bit B → covers m8,10,12,14
m(8,12)+m(10,14): 1-00 & 1-10	1--0	Same as above (duplicate)
m(10,11)+m(14,15): 101- & 111-	1-1-	Differ in bit B → covers m10,11,14,15
m(10,14)+m(11,15): 1-10 & 1-11	1-1-	Same as above (duplicate)

After removing duplicates, the distinct size-4 implicants are:

- I1: 10-- covers m(8,9,10,11) → AB' (A=1, B=0, C & D irrelevant)

- I2: 1--0 covers m(8,10,12,14) → AD' (A=1, D=0, B & C irrelevant)
- I3: 1-1- covers m(10,11,14,15) → AC (A=1, C=1, B & D irrelevant)

Size-4 implicants cannot be combined further. Also check m4:

- m(4,12) = -100 → BC'D' (B=1,C=0,D=0,A irrelevant) — this is a prime implicant covering m4 and m12.

Step 5: Prime Implicant Chart (columns = minterms where F=1 only, NOT d terms):

Prime Implicant	m4	m8	m10	m11	m12	m15
BC'D' = m(4,12)	X				X	
AB' = m(8,9,10,11)		X	X	X		
AD' = m(8,10,12,14)		X	X		X	
AC = m(10,11,14,15)			X	X		X

Step 6: Identify Essential Prime Implicants:

- m4: covered only by BC'D' → BC'D' is an EPI ✓
- m11: covered only by AB' and AC → both cover m11, but m11 alone doesn't determine EPI. Check m8: covered only by AB' and AD'. Check m15: covered only by AC. → AC is an EPI ✓
- After selecting BC'D' and AC, check coverage: m4 ✓(BC'D'), m8(not yet), m10 ✓(AC covers, check), m11 ✓(AC), m12 ✓(BC'D'), m15 ✓(AC).
- m8 still uncovered. AB' covers m8 (and m10,m11 already covered). So add AB'.

Final Minimal Expression: F = BC'D' + AB' + AC

Verification: m4(0100): BC'D'=1·1·1=1 ✓ m8(1000): AB'=1·1=1 ✓ m10(1010): AB'=1·1=1 ✓ m11(1011): AB'=1·1=1 ✓ m12(1100): BC'D'=1·1·1=1 ✓ m15(1111): AC=1·1=1 ✓

6. Quine-McCluskey with Don't Care Terms

6.1 Handling Don't Cares in the QM Method

Don't care terms (denoted as 'd' or 'X') arise when certain input combinations either never occur or when the output is irrelevant. The QM method handles don't cares with a specific rule:

> RULE: Include don't care minterms in PHASE 1 (implicant table) so they can help enlarge prime implicants. BUT exclude don't care minterms from PHASE 2 (prime implicant chart) — they are not required to be covered.

Detailed rules for don't cares in QM:

- PHASE 1: Add all don't care minterms to the groups alongside the regular minterms (F=1). Include them in all comparisons. This allows don't cares to help combine and enlarge implicants.
- When a don't care minterm gets ticked/used in a combination, that's fine — it just means it helped generate a larger implicant.
- PHASE 2: Set up the PI chart with ONLY the regular minterms (F=1) as column headers. Don't care minterms are NOT placed as columns — they don't need to be covered.
- Select EPIs and additional PIs to cover all F=1 minterms only.

The rationale: Don't cares give us freedom to assign them as 0 or 1. We assign them as 1 wherever it helps us form larger groups (better simplification). But we don't need to guarantee the circuit produces any specific output for them.

6.2 Worked Example with Don't Cares

Minimize: F(A,B,C,D) = Σm(2,3,7,9,11,13) + d(1,10,15)

Step 1: List all minterms (F=1) and don't cares in binary:

Minterm	Type	Binary (ABCD)	# 1s
m1	d	0001	1
m2	F=1	0010	1
m3	F=1	0011	2
m7	F=1	0111	3
m9	F=1	1001	2
m10	d	1010	2
m11	F=1	1011	3
m13	F=1	1101	3
m15	d	1111	4

Step 2: Group ALL (including d terms) by number of 1s:

- Group 1 (one 1): m1(0001), m2(0010)
- Group 2 (two 1s): m3(0011), m9(1001), m10(1010)
- Group 3 (three 1s): m7(0111), m11(1011), m13(1101)
- Group 4 (four 1s): m15(1111)

Step 3: Compare adjacent groups to form size-2 implicants:

Pair	Result	Notes
m(1,3)	00-1	G1 vs G2: differ in bit C
m(1,9)	_001 → -001	G1 vs G2: differ in bit A
m(2,3)	001-	G1 vs G2: differ in bit D
m(2,10)	_010/-010: 0010 vs 1010 → -010	Differ in A
m(3,7)	0-11	G2 vs G3: differ in bit B
m(3,11)	_011: 0011 vs 1011 → -011	Differ in A
m(9,11)	10-1	Differ in C
m(9,13)	1-01	Differ in B
m(10,11)	101-	Differ in D
m(7,15)	_111: 0111 vs 1111 → -111	Differ in A
m(11,15)	1-11	Differ in B
m(13,15)	11-1	Differ in C

Step 4: Form size-4 implicants from compatible size-2 pairs:

- m(1,3)+m(9,11): 00-1 & 10-1 → -0-1 → covers m1,m3,m9,m11 → B'D
- m(1,9)+m(3,11): -001 & -011 → -0-1 → same as above (B'D)
- m(3,7)+m(11,15): 0-11 & 1-11 → --11 → covers m3,m7,m11,m15 → CD
- m(9,11)+m(13,15): 10-1 & 11-1 → 1--1 → covers m9,m11,m13,m15 → AD
- m(3,11)+m(7,15): -011 & -111 → --11 → same as CD above

Prime Implicants identified (unchecked items):

- PI1: -010 = B'CD' (covers m2, m10)
- PI2: 001- = A'B'C (covers m2, m3)
- PI3: -0-1 = B'D (covers m1,m3,m9,m11)
- PI4: --11 = CD (covers m3,m7,m11,m15)
- PI5: 1--1 = AD (covers m9,m11,m13,m15)
- PI6: 11-1 = ABD (covers m13, m15)

Step 5: Build PI Chart (columns = F=1 minterms only: 2,3,7,9,11,13):

PI	m2	m3	m7	m9	m11	m13
-010 = B'CD'	X					
001- = A'B'C	X	X				
-0-1 = B'D		X		X	X	
--11 = CD		X	X		X	
1--1 = AD				X	X	X

Step 6: Select EPIs:

- m7: covered only by CD → CD is an EPI ✓
- m13: covered only by AD → AD is an EPI ✓
- After selecting CD and AD: m7 ✓, m9 ✓, m11 ✓, m13 ✓
- Remaining uncovered: m2, m3
- m2 can be covered by B'CD' or A'B'C. m3 can be covered by B'D, CD (already selected), A'B'C.
- CD already selected covers m3 ✓. For m2: select A'B'C (covers both m2 and m3) or B'CD'.
- A'B'C covers m2 and m3 — both remaining. Select A'B'C.

Final Minimal Expression: F = CD + AD + A'B'C

Verification: m2(0010): A'B'C=1·1·1=1 ✓ m3(0011): CD=1·1=1 ✓ m7(0111): CD=1·1=1 ✓ m9(1001): AD=1·1=1 ✓ m11(1011): AD=1·1=1, CD=1·1=1 ✓ m13(1101): AD=1·1=1 ✓

7. Quick Revision – Exam Ready Summary

This section consolidates all key concepts, formulas, and patterns for rapid revision before VTU examinations. Memorize the tables and patterns below.

7.1 Key Definitions

Term	Definition
Combinational Circuit	Logic circuit with only gates (no memory); output depends only on current inputs.
Sequential Circuit	Has memory elements; output depends on current inputs AND stored state.
Minterm (m_i)	Product (AND) of all n variables; each var appears once, true or complemented. Equals 1 for exactly one input combination.
Maxterm (M_i)	Sum (OR) of all n variables. Equals 0 for exactly one input combination. M_i = (m_i)'.
Canonical SoP	Sum (OR) of all minterms where F=1. Each term has ALL literals.
Canonical PoS	Product (AND) of all maxterms where F=0. Each term has ALL literals.
Standard SoP	Simplified SoP; terms may have fewer than n literals.
Standard PoS	Simplified PoS; terms may have fewer than n literals.
K-Map	Graphical grid for Boolean simplification; adjacent cells differ in 1 bit.
Prime Implicant (PI)	A maximal group of 1s (power of 2) in a K-Map that cannot be enlarged further.
Essential PI (EPI)	A PI that alone covers at least one minterm; must be in the minimal expression.
Don't Care (d/X)	Output is irrelevant; can be used as 0 or 1 in grouping to maximize simplification.
Implicant	Any valid group of 1s (power of 2) in K-Map or QM table.
QM Method	Systematic tabular minimization; works for any n variables; computer-implementable.

7.2 Boolean Algebra Quick Reference

#	Rule	What it means
1	A+0=A	Identity for OR
2	A+1=1	Annihilator for OR
3	A·0=0	Annihilator for AND
4	A·1=A	Identity for AND
5	A+A=A	Idempotent (OR)

#	Rule	What it means
6	A+A'=1	Complement (OR) — always 1
7	A·A=A	Idempotent (AND)
8	A·A'=0	Complement (AND) — always 0
9	(A')'=A	Double complement / Involution
10	A+AB=A	Absorption — extra AND-product vanishes
11	A+A'B=A+B	Simplification — complement term drops
12	(A+B)(A+C)=A+BC	Factoring in OR form
DM 1	(XY)'=X'+Y'	DeMorgan: NAND = OR of complements
DM 2	(X+Y)'=X'·Y'	DeMorgan: NOR = AND of complements

7.3 K-Map Grouping Summary

# Variables	# Cells	Group Size → Literals Eliminated	Common Wrap-Around Adjacencies
2	4	2→1, 4→0 (F=1)	Left/right cols adjacent
3	8	2→1, 4→2 vars eliminated, 8→F=1	Top/bottom rows; left/right cols (Gray)
4	16	2→3, 4→2, 8→1, 16→F=1	Top↔Bottom rows; Left↔Right cols; 4 corners form group

K-Map Variable Elimination Rule: In a group of 2^k cells, exactly k variables change — those k variables are ELIMINATED from the product term. The remaining (n-k) variables that are CONSTANT across all cells form the product term.

7.4 QM Method — 10 Steps at a Glance

Step	Action
1	List all F=1 minterms AND don't-care minterms in binary.
2	Count number of 1-bits in each minterm (Hamming weight).
3	Group minterms by their bit-count into groups G0, G1, G2, ...
4	Compare G_k vs G_{k+1}: combine pairs differing in exactly 1 bit. Write '-' for differing bit. Tick used minterms.
5	Repeat Step 4 on new implicants. Ensure '-' positions match when combining size-4 or larger implicants.
6	Unticked implicants = PRIME IMPLICANTS. List them.
7	Build PI Chart: Rows=PIs, Columns=F=1 minterms ONLY (exclude d terms).
8	Mark X where each PI covers each minterm.

Step	Action
9	Find EPIs: columns with only one X. Must include their PI. Mark all minterms they cover.
10	Add minimum additional PIs to cover remaining uncovered minterms. Write final OR of selected PIs.

7.5 Common Mistakes & VTU Exam Patterns

Common Mistakes to Avoid

- Forgetting Gray code ordering in K-Maps: columns must go 00,01,11,10 — NOT 00,01,10,11.
- Using non-power-of-2 group sizes in K-Maps (e.g., groups of 3 or 6 are INVALID).
- Including a 0 cell in a K-Map group — only 1s and don't-cares may be grouped.
- Forgetting wrap-around adjacencies — corner cells and edge cells can wrap around.
- Missing Essential Prime Implicants — always check EPIs first before adding other PIs.
- In QM: including don't-care minterms as columns in the PI Chart (Phase 2) — they must not appear there.
- In QM: combining implicants with mismatched dash positions — dashes must align exactly.
- Writing maxterms incorrectly — for a 0 input, the variable appears TRUE in maxterm (opposite of minterm rule).
- Confusing standard SoP (simplified) with canonical SoP (full, all literals) — they are different.

Frequently Asked VTU Exam Questions

Common VTU Question Type	Marks (Typical)
Define combinational logic. Compare with sequential logic. Draw block diagrams.	5–10 M
State and prove Laws & Rules of Boolean Algebra (esp. Rules 10,11,12).	5–10 M
State and verify DeMorgan's Theorems using truth tables.	5 M
Convert given function to Canonical SoP / PoS form from truth table.	5 M
Convert canonical to standard SoP/PoS form with step-by-step simplification.	5–10 M
Simplify Boolean function using K-Map (3 or 4 variables), show all groups.	10 M
K-Map with don't care conditions — identify EPIs and give minimal expression.	10 M
Minimize using Quine-McCluskey method — show full PI table and PI chart.	10 M
QM method with don't care terms — full worked example.	10 M

Formula Quick-Reference Card

SoP → look for 1s in output | PoS → look for 0s in outputMinterm: 0→complemented, 1→true | Maxterm: 0→true, 1→complementedF = Σm(list of 1-rows) = ΠM(list of 0-rows)K-Map group of

2^k cells eliminates k variablesQM: d terms included in Phase 1, excluded from Phase 2EPI = PI that alone covers ≥1 minterm → always includeDeMorgan 1: (AB)' = A'+B' | DeMorgan 2: (A+B)' = A'·B'

BEC302 Module 1 | Prepared for VTU Students | vtu-easy.com

All the best for your examinations!